

Distributed File Storage

Autor: Cristian A. Silva

1. Introducción

El proyecto **Distributed File Storage (DFS)** implementa un sistema de almacenamiento distribuido compuesto por cuatro tipos de componentes:

- **Cliente**
- **NameNode**
- **DataNode**
- **Logger centralizado**

El objetivo principal es permitir que un usuario pueda **subir, descargar y consultar archivos** a través de un cliente, mientras el sistema se encarga de:

- Partir los archivos en bloques.
- Distribuir estos bloques entre distintos DataNodes.
- Registrar metadatos y ubicación de los bloques en el NameNode.
- Mantener un registro centralizado de eventos y operaciones mediante el Logger.

En este informe se describen los principales archivos y procesos de cada componente.

2. Cliente

El cliente es el componente que **interactúa con el usuario y se comunica con el NameNode y los DataNodes**. Puede verse como la interfaz de línea de comandos (CLI) del sistema.

2.1. Archivos del cliente

- `cmd/client/main.go`
 - `internal/client/client.go`
-

2.2. `cmd/client/main.go`

Este archivo contiene la función `main` del cliente del sistema de almacenamiento distribuido. Sus responsabilidades son:

- Interpretar los **argumentos de la línea de comandos**.
- Inicializar el **sistema de logging centralizado**.
- Crear una **instancia del cliente TCP**.
- Despachar las operaciones pedidas por el usuario (`ping`, `ls`, `info`, `put`, `get`) hacia la lógica del paquete `client`.

En otras palabras, `main.go` es una capa fina que traduce los comandos que escribe el usuario en la consola en llamadas a métodos de alto nivel del cliente.

2.2.1. Comandos soportados

El programa se ejecuta con la forma:

```
client <comando> [parámetros...]
```

Los comandos principales son:

- `ping`
- `ls`
- `info <remoteName>`
- `put <localPath> <remoteName>`
- `get <remoteName> <localPath>`

Si no se pasa ningún comando o los parámetros son incorrectos, el cliente muestra un mensaje de uso indicando la forma correcta.

2.2.2. `ping`

Comando sencillo para comprobar si el cliente puede interactuar con el sistema.

- El `main` llama a `c.Ping()`.
- El cliente envía un comando `PING` al `NameNode`.
- El `NameNode` responde, típicamente, con algo tipo `"PONG"` (o similar), que se registra en el log.

Sirve como prueba rápida de conectividad y funcionamiento básico.

2.2.3. `put`

Fragmento principal en `main.go`:

```
if len(os.Args) != 4 {  
    log.Fatal("uso: client put <localPath> <remoteName>")  
}  
local := os.Args[2]  
remote := os.Args[3]  
  
if err := c.Put(local, remote); err != nil {  
    // manejo de error  
}
```

Este comando:

- **Sube un archivo** desde `localPath` al DFS con nombre lógico `remoteName`.
- El `main` se limita a:
 - Validar parámetros.
 - Llamar a `c.Put`.
 - Manejar/loguear errores.

La lógica real de particionar el archivo y enviarlo a los DataNodes está en `internal/client/client.go`.

2.2.4. `ls`

Fragmento en `main.go`:

```
files, err := c.List()
if err != nil {
    // manejo de error
}

if len(files) == 0 {
    fmt.Println("No hay archivos en el namenode.")
} else {
    fmt.Println("Archivos en el namenode:")
    for _, f := range files {
        fmt.Println(" -", f)
    }
}
```

Este comando:

- Llama a `c.List()` para obtener la **lista de archivos registrados** en el sistema.
- Muestra la lista por pantalla.
- Internamente se registra en el log la cantidad de archivos obtenidos.

2.2.5. info

```
if len(os.Args) != 3 {
    log.Fatal("uso: client info <remoteName>")
}
remote := os.Args[2]
if err := c.Info(remote); err != nil {
    // manejo de error
}
```

- Obtiene información detallada de un archivo determinado, por ejemplo:
 - Metadatos.
 - Ubicación de los bloques.
 - Otros datos que exponga el NameNode.
- La lógica de comunicación y parseo está en `client.go`.

2.2.6. get

```
if len(os.Args) != 4 {
    log.Fatal("uso: client get <remoteName> <localPath>")
}
remote := os.Args[2]
local := os.Args[3]

if err := c.Get(remote, local); err != nil {
    // manejo de error
}
```

Este comando:

- **Descarga un archivo** del DFS (identificado por `remoteName`) y lo guarda en la ruta local `localPath`.
 - `c.Get` se encarga de:
 - Consultar al NameNode dónde están los bloques de ese archivo.
 - Descargar los bloques desde los DataNodes.
 - Reconstruir el archivo en el cliente.
-

2.3. `internal/client/client.go`

Este archivo contiene la **lógica principal del cliente**. Se encarga de:

- Dividir archivos en bloques al subirlos.
- Enviar esos bloques a los DataNodes correspondientes.
- Descargar los bloques de los DataNodes y recomponer archivos.
- Definir el protocolo de comunicación con:
 - El **NameNode**: comandos `PING`, `PUT`, `GET`, `INFO`, `LS`.
 - Los **DataNodes**: operaciones `WRITE` y `READ` de bloques.

2.3.1. Estructuras `PutPlan` y `BlockPlan`

```
type PutPlan struct {
    FileName  string    `json:"file_name"`
    BlockSize int       `json:"block_size"`
    Blocks    []BlockPlan `json:"blocks"`
    Status    string    `json:"status"`
}

type BlockPlan struct {
    BlockID  string `json:"block_id"`
    Replicas []string `json:"replicas"`
}
```

- **PutPlan** representa el **plan de distribución de bloques** que devuelve el NameNode cuando el cliente quiere subir un archivo. Incluye:
 - **FileName**: nombre lógico del archivo en el DFS.
 - **BlockSize**: tamaño de cada bloque en bytes.
 - **Blocks**: lista de bloques del archivo.
 - **Status**: estado de la operación (por ejemplo, "OK" si todo está bien).
- **BlockPlan** describe un **bloque individual**:
 - **BlockID**: identificador único del bloque.
 - **Replicas**: lista de DataNodes donde debe almacenarse el bloque.

2.3.2. Interfaz **Client**

```
type Client interface {
    Put(localPath string, remoteName string) error
    Get(remoteName string, localPath string) error
    Info(remoteName string) error
    List() ([]string, error)
    Ping() error
}
```

Define las operaciones de alto nivel que el cliente expone al resto del sistema (por ejemplo, a `main.go`).

2.3.3. Implementación concreta: **tcpClient**

```
type tcpClient struct {
    addr string
}

func NewTCPClient(addr string) Client {
    return &tcpClient{addr: addr}
}
```

`addr` guarda la **dirección del NameNode** (por ejemplo, `"localhost:3000"`).
`NewTCPClient` crea una implementación concreta de `Client` que usa TCP.

2.3.4. Método `sendCommand`

```
func (c *tcpClient) sendCommand(line string) (string, error) {
    conn, err := net.Dial("tcp", c.addr)
    ...
    fmt.Fprintf(conn, "%s\n", line)
    response, err := bufio.NewReader(conn).ReadString('\n')
    ...
    return strings.TrimSpace(response), nil
}
```

- Abre una conexión TCP al NameNode.
- Envía una línea de texto que representa un comando del protocolo (por ejemplo `"PING"`, `"PUT archivo.txt 1024"`, `"LS"`).
- Lee una línea de respuesta.
- Es la base de todos los comandos que van al NameNode.

2.3.5. `Ping()`

```
func (c *tcpClient) Ping() error {
    resp, err := c.sendCommand("PING")
    ...
    dfslog.Infof("Respuesta PING del namenode (%s): %s", c.addr,
resp)
    return nil
}
```

- Envía el comando `PING` al NameNode.
- Registra en el log la respuesta recibida.

2.3.6. `Put(localPath, remoteName)`

Este método implementa todo el flujo de subida:

1. **Lectura del archivo local.**
2. **Solicitud de plan al NameNode:** se envía un comando **PUT** con el nombre lógico del archivo y su tamaño; el NameNode responde con un **PutPlan** en formato JSON.
3. **División en bloques:** según **BlockSize** y la cantidad de bloques definida en **PutPlan**.
4. **Envío de cada bloque a los DataNodes** indicados en **Replicas**.

Ejemplo de función auxiliar para escribir en un DataNode:

```
func storeToDataNode(addr, blockID string, data []byte) error {
    conn, _ := net.Dial("tcp", addr)
    defer conn.Close()

    header := fmt.Sprintf("WRITE %s %d\n", blockID, len(data))
    conn.Write([]byte(header))
    conn.Write(data)

    ack, _ := bufio.NewReader(conn).ReadString('\n')
    if strings.TrimSpace(ack) != "OK" {
        return fmt.Errorf("datanode %s respondió error: %s", addr,
ack)
    }
    return nil
}
```

- Se envía el comando **WRITE** con el **blockID** y el tamaño de los datos.
- Luego se envían los bytes del bloque.
- Se espera la respuesta **"OK"** para confirmar que el bloque fue guardado.

2.3.7. **Get(remoteName, localPath)**

Para la descarga de un archivo, el flujo es inverso:

1. El cliente envía un comando **GET** al NameNode para obtener un **PutPlan** con la lista de bloques y sus réplicas.

2. Para cada bloque:

- Se prueba leer desde cada réplica hasta que alguna responda correctamente.
- Los bloques se van concatenando en un buffer en el orden original.

3. Finalmente, se escribe el archivo reconstruido en `localPath`.

Ejemplo de fragmento:

```
var buf bytes.Buffer

for _, b := range plan.Blocks {
    for _, addr := range b.Replicas {
        blockData, lastErr = readFromDataNode(addr, b.BlockID)
        if lastErr == nil {
            break
        }
    }
    if lastErr != nil {
        // manejo de error
    }
    buf.Write(blockData)
}
```

Función auxiliar para leer de un DataNode:

```
func readFromDataNode(addr, blockID string) ([]byte, error) {
    conn, _ := net.Dial("tcp", addr)
    defer conn.Close()

    fmt.Fprintf(conn, "READ %s\n", blockID)
    header, _ := reader.ReadString('\n')

    if !strings.HasPrefix(header, "DATA ") { ... }

    size, _ := strconv.Atoi(strings.TrimPrefix(header, "DATA "))
    data := make([]byte, size)
    io.ReadFull(reader, data)
    return data, nil
}
```

- Se solicita al DataNode que retorne un bloque con el comando `READ`.
- Se valida que la respuesta comience con `DATA`.
- Se leen exactamente `size` bytes correspondientes al bloque.

2.3.8. `Info(remoteName)` y `List()`

- `Info(remoteName)`: envía el comando `INFO` al NameNode y, si el archivo existe, loguea la metadata asociada.
 - `List()`: envía el comando `LS` al NameNode y devuelve la lista de nombres de archivos registrados.
-

3. NameNode

El **NameNode** es el componente que guarda la **metadata** del sistema. No almacena los bloques de los archivos, pero sí sabe:

- Qué archivos existen.
- En qué bloques están divididos.
- En qué DataNodes se ubica cada bloque.

Es, en cierto sentido, el “organizador” del DFS y es central para su funcionamiento.

3.1. Interfaz del NameNode

A alto nivel, el NameNode expone métodos como:

- `HandlePut(filename string, sizeBytes int)`
 - Planifica dónde irán los bloques de un archivo nuevo.
 - Devuelve, para cada bloque, su ID y la lista de DataNodes donde deben guardarse.

- `HandleGet(filename string) ([]metadata.BlockLocation, bool, error)`
 - Devuelve el plan de bloques guardado para un archivo.
 - El `bool` indica si el archivo existe en el store.
- `HandleInfo(filename string) ([]metadata.BlockLocation, bool, error)`
 - Similar a `HandleGet`, pero podría extenderse para devolver metadatos adicionales.
- `HandleList() ([]string, error)`
 - Devuelve la lista de archivos conocidos por el NameNode.

3.2. Métodos auxiliares

- `pickDataNodes(n int)`
 - Devuelve hasta `n` DataNodes distintos para asignar las réplicas de un bloque.

4. Metadata

El funcionamiento del NameNode se apoya sobre un módulo denominado **metadata**, que representa la información de metadatos del DFS.

4.1. Interfaz **Store**

```
type Store interface {
    Load() error
    Save() error

    // consulta
    ListFiles() ([]string, error)
    GetFile(name string) ([]BlockLocation, bool)

    // modificación
    PutFile(name string, locations []BlockLocation) error
}
```

```
    DeleteFile(name string) error
}
```

- `Store` representa el mecanismo de almacenamiento de metadata.
- Se puede implementar de distintas formas. En este proyecto, se implementa una versión basada en JSON.

4.2. Implementación JSON

- `json_storage` es una implementación concreta de `Store` que persiste la información en un archivo JSON.
 - Esto permite:
 - Guardar el estado del NameNode en disco.
 - Restaurar la información de archivos y bloques al reiniciar el sistema.
-

5. DataNode

El **DataNode** es el componente encargado de **almacenar físicamente los bloques**. Su función es recibir bloques desde el cliente (cuando se hace un `put`) y devolverlos (cuando se hace un `get`).

5.1. Interfaz `DataNode`

```
type DataNode interface {
    StoreBlock(blockID string, data []byte) error
    RetrieveBlock(blockID string) ([]byte, error)
}
```

- `StoreBlock(blockID, data)`: guarda un bloque identificado por `blockID`.
- `RetrieveBlock(blockID)`: devuelve los datos del bloque con el ID indicado.

5.2. Implementación en memoria

```
type memoryDataNode struct {
    mtx    sync.RWMutex
```

```
    blocks map[string][]byte  
}
```

- `memoryDataNode` encapsula la relación entre IDs de bloques y sus datos.
 - Usa un `sync.RWMutex` para evitar problemas de **sincronización** cuando múltiples operaciones intentan leer o escribir bloques al mismo tiempo.
 - En esta implementación, los bloques se guardan en memoria, lo cual es suficiente para pruebas y desarrollo.
-

6. Logger

El **Logger** es el componente responsable de llevar un **registro centralizado del estado del sistema**.

- Se ejecuta en un proceso aparte, que abre un **socket TCP**.
 - El cliente, el NameNode y los DataNodes se conectan a este servidor de logs para:
 - Enviar información de estado.
 - Registrar operaciones importantes (PUT, GET, errores, etc.).
 - Esta arquitectura de logging centralizado permite:
 - Depurar problemas de manera más sencilla.
 - Tener una vista global de la actividad del DFS.
-

7. Conclusión

El proyecto **Distributed File Storage** implementa una arquitectura clásica de DFS, separando claramente responsabilidades:

- El **Cliente** ofrece una interfaz de línea de comandos simple y delega la lógica compleja en `client.go`.
- El **NameNode** mantiene la metadata y decide cómo distribuir los bloques.

- Los **DataNodes** almacenan y recuperan datos siguiendo un protocolo simple (WRITE/READ).
- El **Logger** proporciona trazabilidad y facilita el monitoreo del sistema.

La combinación de estos componentes permite subir y descargar archivos de forma distribuida, manteniendo un control centralizado de la metadata y dejando la parte pesada de almacenamiento a los DataNodes.